

BUILD TO UNDERSTAND

Seven Steps in a Repository

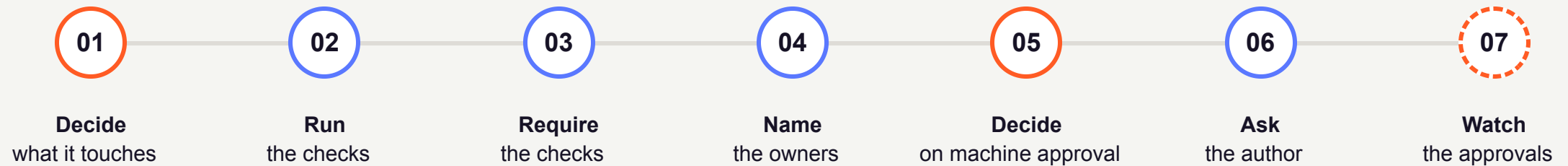
Seven steps, in the order I would do them. Every one is a file in the repository or a setting on it, except the last, which is half an hour a month.

From the exploration *What Is the Second Pair of Eyes For?*



THE AGENDA

Seven steps, in the order they get done



○ Two are decisions you write down ○ Four you configure once ○ One that repeats

Everything on this track is a file in the repository or a setting on it, except the last, which is half an hour a month. **Nothing here needs a platform team, a budget, or a vendor.**

Decide what this repository touches

Nothing settles this for you. Whether the repository moves money or holds customer data decides how strict every step after this one should be, so it goes first.

- Does it move money, hold customer data, or neither?
- Write the answer in the README, under “Review rules”, with the date
- Say what follows: reviewers, machine approval, security review
- Revisit it when the repository starts touching something new

```
## Review rules
```

```
Touches: customer money, through  
settlement and the ledger. 2026-09-28
```

```
What follows: two reviewers on every  
change, security review required,  
no auto-merge.
```

One dated sentence in the README, **where a new joiner will find it**.

Make the mechanical checks run on every change

Everything mechanically verifiable runs before a person looks at the change.

- Formatting and linting, in a workflow file under `.github/workflows/`
- The test suite, in the same file
- A static analysis pass, in the same file
- Dependency scanning, in `.github/dependabot.yml`
- Secret scanning, a toggle in the repository settings

Google's own reason: static analysis exists so that human review can **“focus on issues that are not mechanically verifiable.”**

Make the checks from step 2 block a merge

Running a check and requiring it are two different settings. A check that runs is a suggestion. A check that blocks a merge is a control.

- Require the formatter, the linter, the tests and the static analysis pass, each by name
- Require a pull request before merging
- Stop direct pushes to the default branch
- Dismiss stale approvals when new commits land

All four live in branch protection, or in a ruleset. **Dismissing stale approvals** makes an approval refer to the code that actually merges.

Name the owners of the paths that carry risk

Step 1 settled what the whole repository touches. Some paths inside it carry more than the rest, and that difference fits in a file.

- List the paths that move money or hold customer data
- Give each a named reviewer or team
- Leave the rest on the lighter default
- Switch on “require review from code owners”, or the file only suggests

```
# .github/CODEOWNERS
# Everything, unless a later line overrides it.
*                @payments-team

# The paths that move money.
/src/settlement/  @settlement-owners @security
/src/ledger/      @settlement-owners

# Anything that changes how the service is deployed.
/.github/workflows/ @platform
```

The **last matching line wins**, so the catch-all goes first and the risky paths underneath it.

Decide whether a machine may approve here

Write it down. A setting with no owner is a decision nobody made. The README is the answer to where, and the reason is the answer to why there.

- Decide whether a machine review counts toward required approvals
- Add it to the same “Review rules” heading step 1 started
- Give the reason, not only the answer
- Revisit it when what the repository touches changes

```
## Review rules
```

```
Touches: customer money, through  
settlement and the ledger. 2026-09-28
```

```
What follows: two reviewers on every  
change, security review required,  
no auto-merge.
```

```
Machine approval: not allowed here. A  
machine approval would be the only  
approval on a change that moves money,  
and nothing logs that it happened.  
2026-09-28
```

The setting has nowhere to hold a reason, and **nothing logs a machine approval**. The README is where the decision survives.

Ask the author for what no check can see

One template, the same questions in front of every change. The second heading is the load-bearing one.

- What changed, and why it belongs here
- What you want looked at, specifically
- How to undo it
- Which parts an agent wrote

```
<!-- .github/pull_request_template.md -->

## What this changes, and why it belongs here


## What I want looked at
<!-- Name one thing. "Anything" is not
      an answer. -->


## How to undo it


## What an agent wrote, and what I checked
```

Across **80.000 pull requests**, descriptions rarely changed the review. The exception explained the code *and* asked for specific feedback.

Watch for the day the approvals stop meaning anything

Steps 1 to 6 can all be in place and still stop working, and nothing announces it. The checks stay green, the approvals keep arriving, the template keeps getting filled in, and none of that requires anyone to have read the change.

Once a month, read ten merged pull requests and ask what you are seeing:

- Approvals that arrived a minute after opening, on changes too long to read in a minute
- Template sections filled with “N/A”, the branch name, or the commit message pasted back
- The same person approving everything, including the parts they do not work on
- Code owners approving paths they have never otherwise touched

No setting reports any of this. **Somebody has to go and look.**

WHAT THE LIST CANNOT DO

**It makes the decision cheaper to reach.
It does not make the decision.**

Steps 2 to 4 make the judgement cheaper to reach. Step 6 gives the decider something to read. Step 7 checks that the decision is still being made. None of them decides whether the change is wanted.

And I have not run this list against our own repositories. I do not know how many of these settings are already on, which makes step 3 the first thing I owe the argument.